Índice

Índice 1
Configuración del entorno 1
Obtención del código fuente de la aplicación 6
Análisis del AndroidManifest.xml 8
Permisos 9
Configuración insegura de la etiqueta <application> 10
Componentes de la aplicación exportados sin protección 11
Análisis del código fuente de la aplicación en busca de vulnerabilidades 12
Almacenamiento inseguro de credenciales mediante criptografía débil 12
ContentProvider exportado sin controles de acceso 14
Análisis de aplicación móvil en dinámico 16
Interceptación del tráfico entre aplicación y servidor. 16
Vulnerabilidades de depuración de la aplicación. 18
Almacenamiento inseguro de datos. 19
Bypass de autenticación 21
Bypass de detección de root. 23
Bypass de detección de emulador. 25

Configuración del entorno

Antes de iniciar el análisis se preparó el entorno de trabajo necesario para la auditoría: comprobación de las herramientas disponibles, creación de un entorno virtual de Python para el servidor de pruebas y despliegue de un emulador Android sobre el que instalar la aplicación.

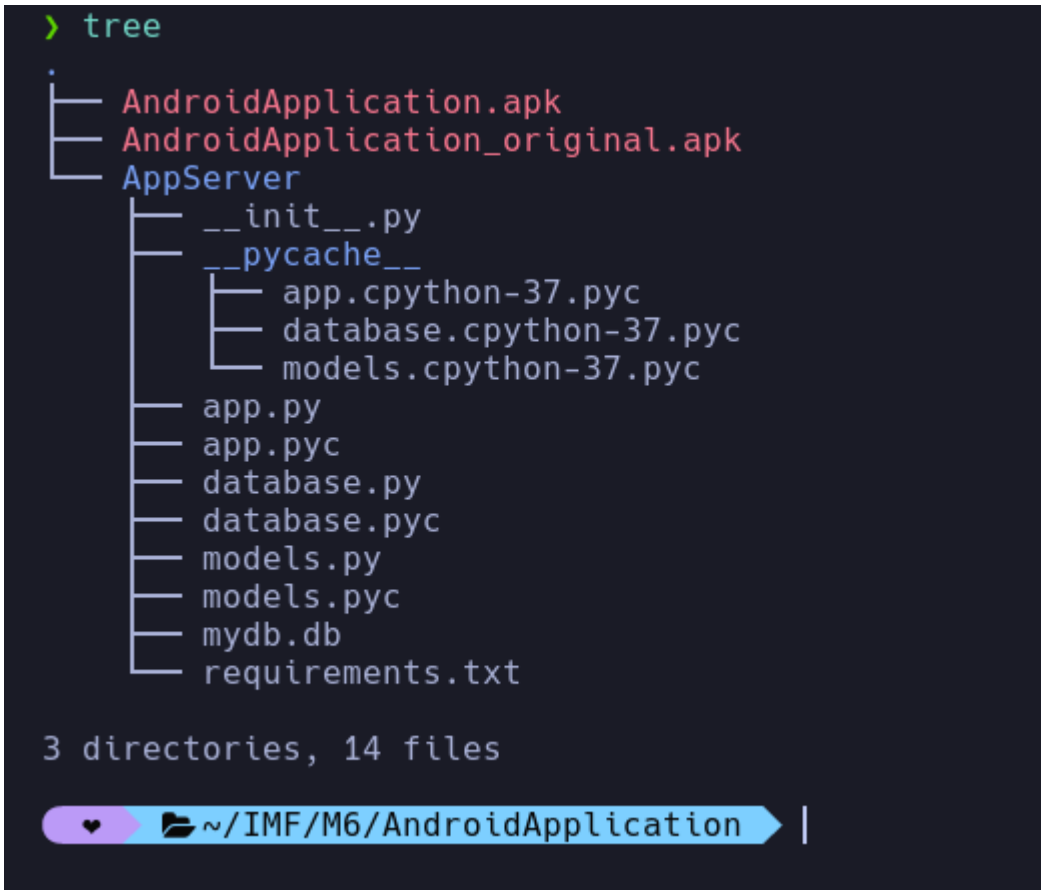


Figura 1. Estructura del proyecto



Figura 2. Identificación del tipo de fichero y cálculo del hash del APK de trabajo



Figura 3. Comprobación de las herramientas de análisis disponibles en el sistema

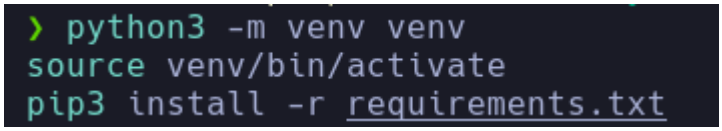


Figura 4. Creación del entorno virtual de Python e instalación de las dependencias del servidor backend

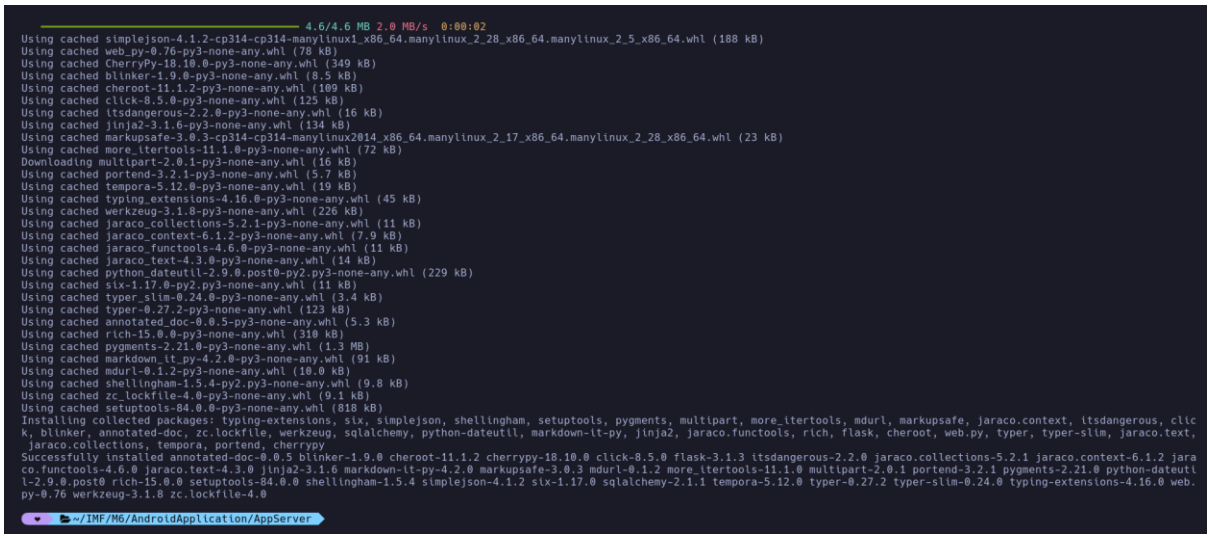


Figura 5. Instalación de las dependencias del backend dentro del entorno virtual.

```
❯ pip3 install "sqlalchemy<1.4" --force-reinstall
Collecting sqlalchemy<1.4
  Using cached sqlalchemy-1.3.24-cp314-cp314-linux_x86_64.whl
Installing collected packages: sqlalchemy
  Attempting uninstall: sqlalchemy
    Found existing installation: SQLAlchemy 2.1.1
    Uninstalling SQLAlchemy-2.1.1:
      Successfully uninstalled SQLAlchemy-2.1.1
Successfully installed sqlalchemy-1.3.24
❯ python3 app.py
The server is hosted on port: 8888
```

Figura 6. Ajuste de la versión de SQLAlchemy (downgrade a <1.4) y arranque del servidor backend en el puerto 8888.

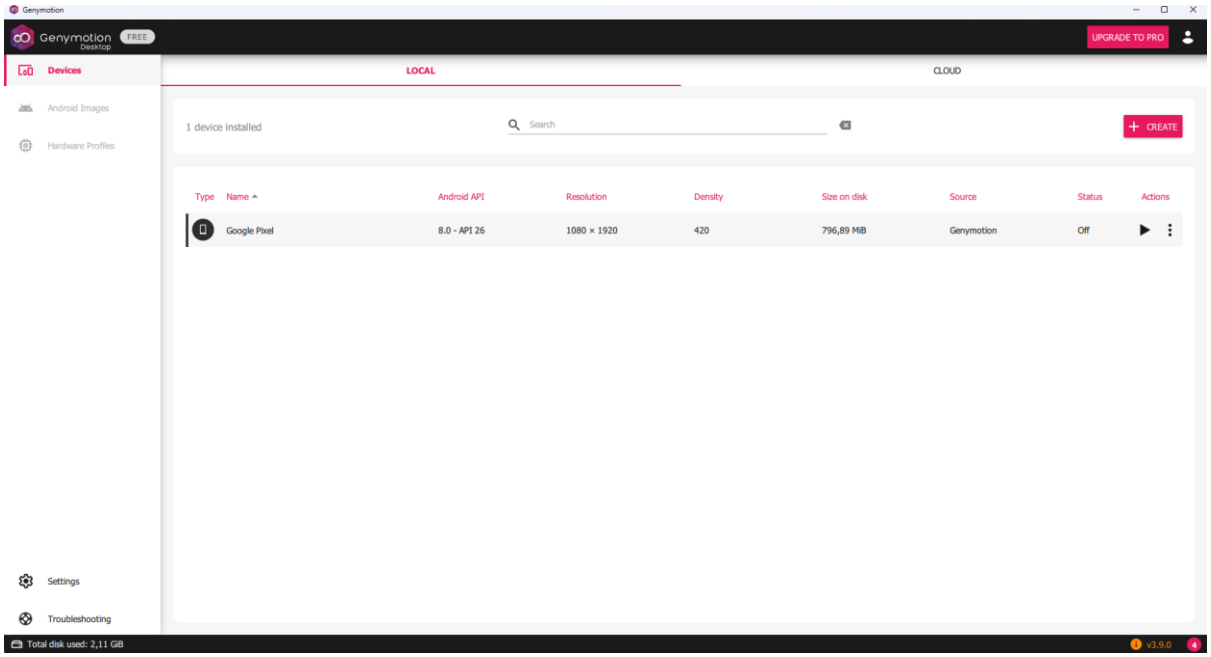


Figura 7. Despliegue de un dispositivo virtual Google Pixel mediante Genymotion.

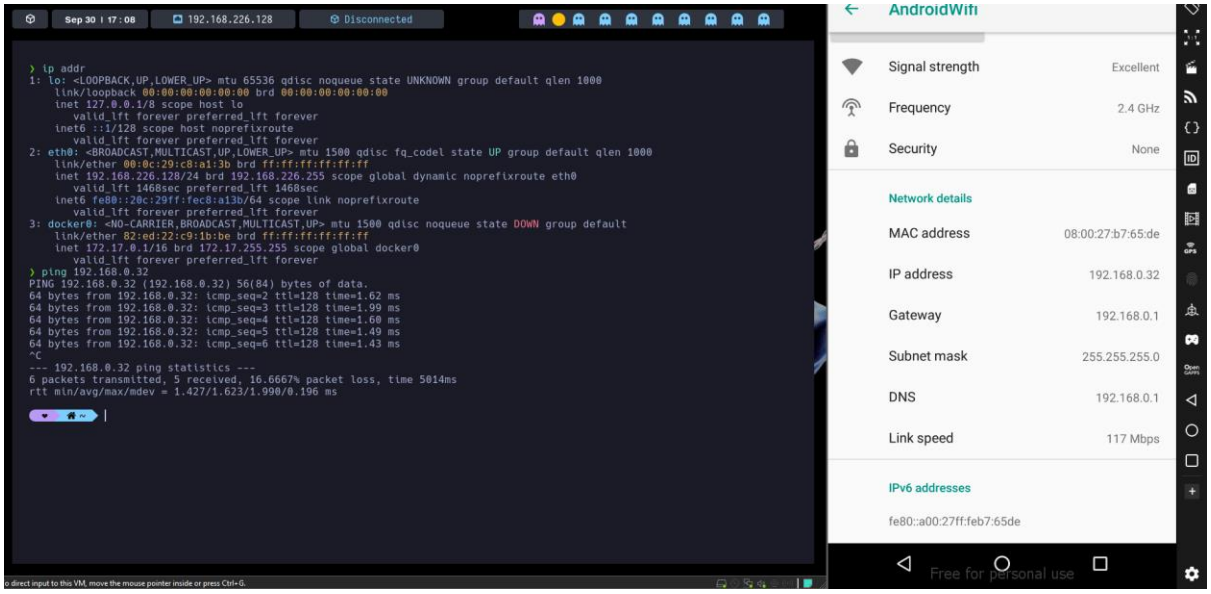


Figura 8. Comprobación de la conectividad de red del emulador y de los datos de red mostrados en el propio dispositivo (IP 192.168.0.32).

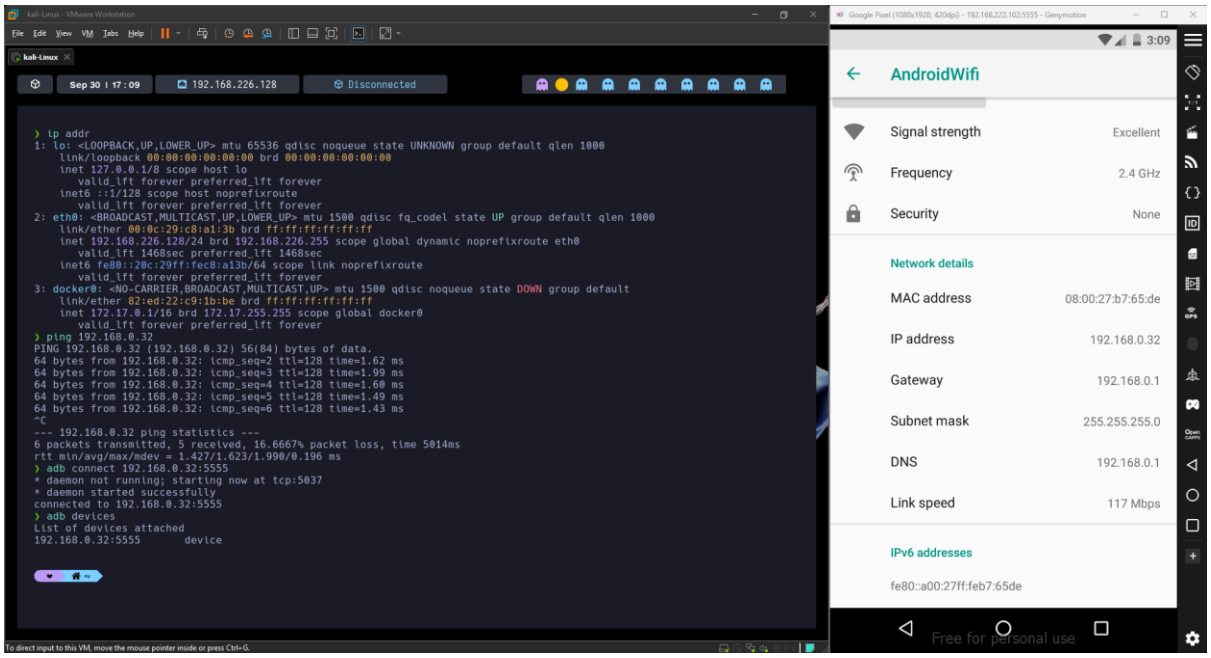


Figura 9. Conexión de adb al emulador por IP (adb connect) y verificación del dispositivo en adb devices

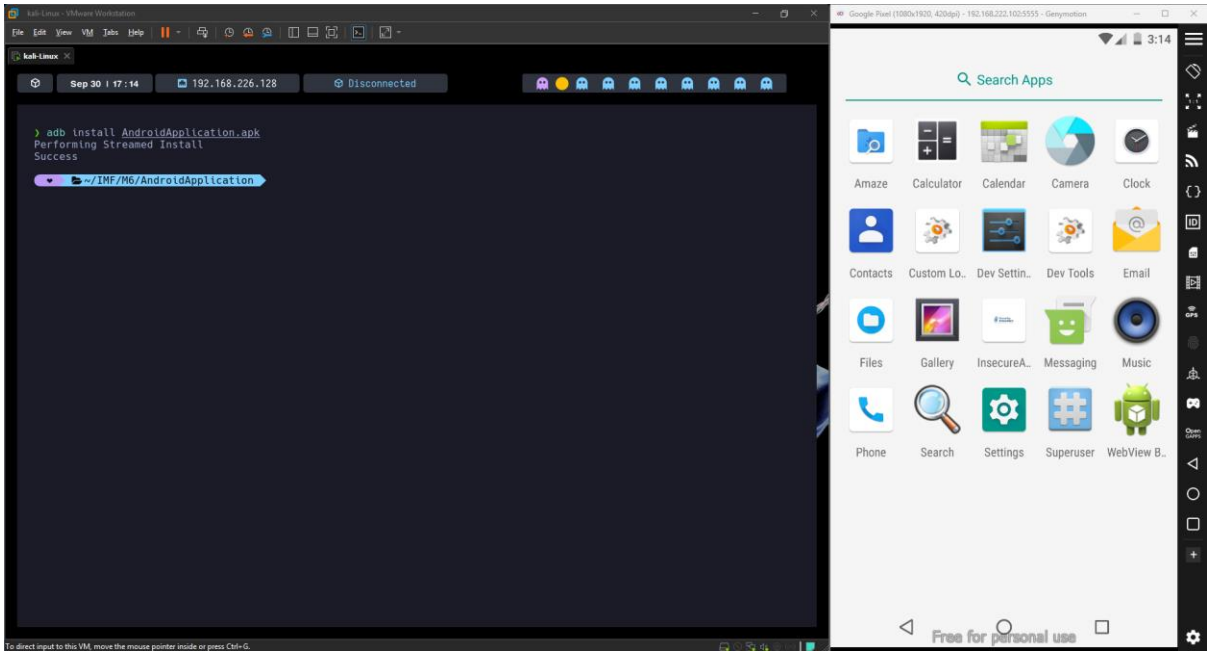


Figura 10. Instalación de la aplicación en el emulador (adb install) y comprobación del icono InsecureApp en el launcher del dispositivo.

Obtención del código fuente de la aplicación

Para obtener el código fuente de la aplicación se ha partido del APK y se han empleado dos herramientas de decompilación distintas, con el fin de disponer tanto de los recursos y el manifest como del código Java legible.

En primer lugar, se ha utilizado apktool para decodificar el APK, obteniendo el AndroidManifest.xml en texto plano, los recursos y el código smali:

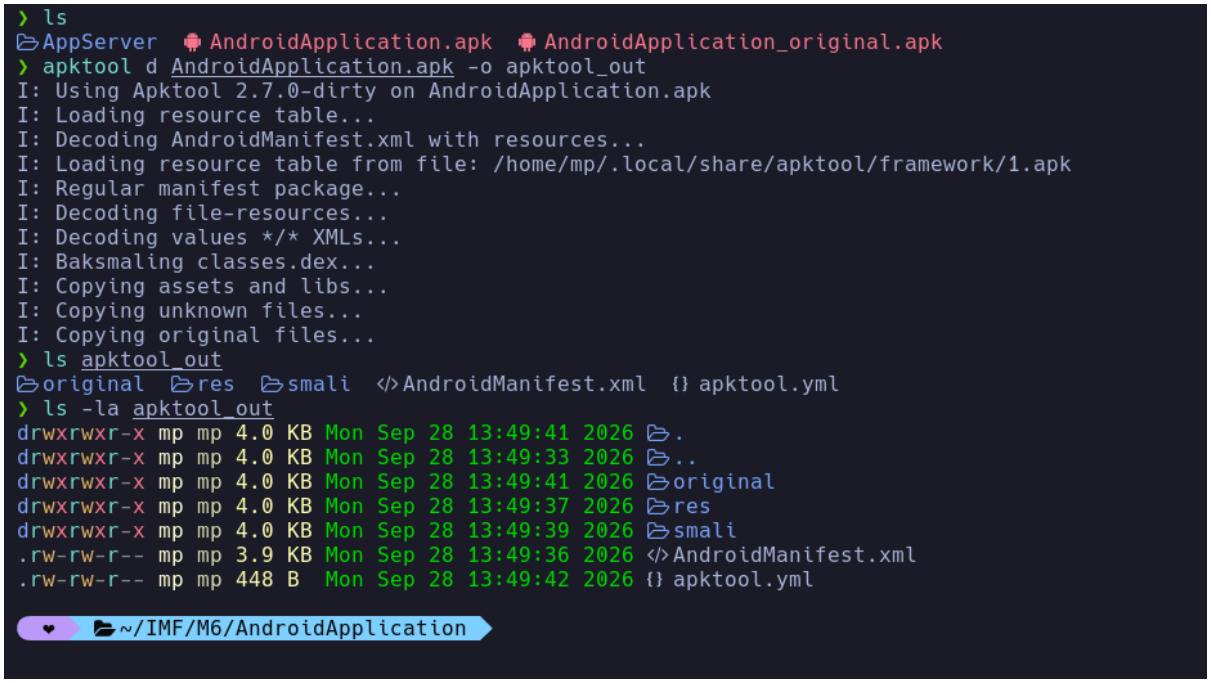


Figura 11. Decodificación del APK con apktool. Se obtienen las carpetas originales, res y smali, además del AndroidManifest.xml ya decodificado.

A continuación, se ha empleado jadx para decompilar directamente el APK a código Java legible:

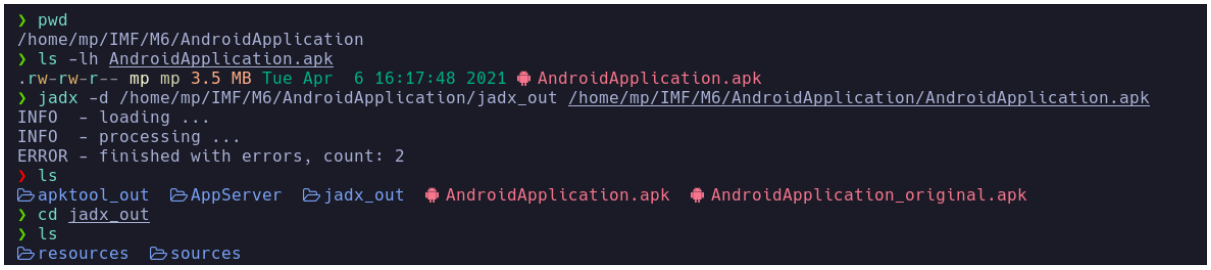


Figura 12. Decompilación del APK con jadx. El proceso finaliza con 2 errores no bloqueantes y genera las carpetas resources/ y sources/.

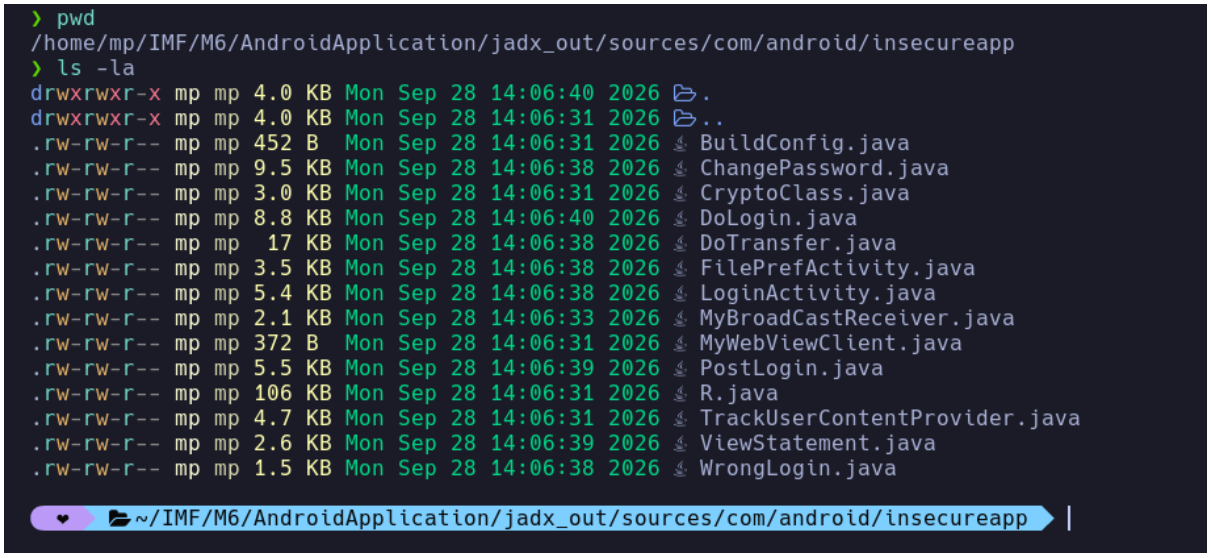


Figura 13. Listado de las clases Java obtenidas en sources/com/android/insecureapp, punto de partida para el análisis de código fuente del apartado 3.

Análisis del AndroidManifest.xml

Una vez decodificado el AndroidManifest.xml con apktool, se ha revisado su contenido completo en busca de permisos, configuración de la aplicación y componentes expuestos.

Contenido relevante del manifest decodificado:

1 | <manifest xmlns:android="http://schemas.android.com/apk/res/android"

| package="com.android.insecureapp">

2 | <uses-permission android:name="android.permission.INTERNET"/>

3 | <uses-permission
android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>

4 | <uses-permission android:name="android.permission.SEND_SMS"/>

5 | <uses-permission android:name="android.permission.USE_CREDENTIALS"/>

6 | <uses-permission android:name="android.permission.GET_ACCOUNTS"/>

7 | <uses-permission android:name="android.permission.READ_PROFILE"/>

8 | <uses-permission android:name="android.permission.READ_CONTACTS"/>

9 | <uses-permission android:name="android.permission.READ_PHONE_STATE"/>

10 | <uses-permission android:maxSdkVersion="18"

| android:name="android.permission.READ_EXTERNAL_STORAGE"/>

11 | <uses-permission android:name="android.permission.READ_CALL_LOG"/>

12 | <uses-permission
android:name="android.permission.ACCESS_NETWORK_STATE"/>

13 | <uses-permission
android:name="android.permission.ACCESS_COARSE_LOCATION"/>

14 | <uses-feature android:glEsVersion="0x00020000" android:required="true"/>

15 | <application android:allowBackup="true" android:debuggable="true"

| android:icon="@mipmap/ic_launcher" android:label="@string/app_name"

| android:theme="@android:style/Theme.Holo.Light.DarkActionBar">

16 | <activity android:name="com.android.insecureapp.LoginActivity">
(LAUNCHER)

22 | <activity android:name="com.android.insecureapp.FilePrefActivity"/>

23 | <activity android:name="com.android.insecureapp.DoLogin"/>

24 | <activity android:exported="true" android:name="...PostLogin"/>

25 | <activity android:name="com.android.insecureapp.WrongLogin"/>

26 | <activity android:exported="true" android:name="...DoTransfer"/>

27 | <activity android:exported="true" android:name="...ViewStatement"/>

28 | <provider
android:authorities="com.android.insecureapp.TrackUserContentProvider"

| android:exported="true" android:name="...TrackUserContentProvider"/>

29 | <receiver android:exported="true" android:name="...MyBroadCastReceiver">

31 | <action android:name="theBroadcast"/>

34 | <activity android:exported="true" android:name="...ChangePassword"/>

| ... (componentes de Google Play Services / Ads, sin relevancia para el análisis)

45 | </manifest>

Permisos

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.SEND_SMS"/>
<uses-permission android:name="android.permission.USE_CREDENTIALS"/>
<uses-permission android:name="android.permission.GET_ACCOUNTS"/>
<uses-permission android:name="android.permission.READ_PROFILE"/>
<uses-permission android:name="android.permission.READ_CONTACTS"/>
<uses-permission android:name="android.permission.READ_PHONE_STATE"/>
<uses-permission android:maxSdkVersion="18" android:name="android.permission.READ_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.READ_CALL_LOG"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
```

Figura 14. Bloque de permisos declarado en el AndroidManifest.xml.

Permiso	Funcionalidad
INTERNET	Permite establecer comunicaciones de red.
WRITE_EXTERNAL_STORAGE	Permite escribir información en el almacenamiento externo.
SEND_SMS	Permite enviar mensajes SMS.
USE_CREDENTIALS	Permite utilizar credenciales almacenadas en el dispositivo.
GET_ACCOUNTS /	Permite acceder a las cuentas configuradas en el dispositivo.
READ_PROFILE	Permite acceder a información del perfil del usuario.
READ_CONTACTS	Permite consultar los contactos almacenados.
READ_PHONE_STATE	Permite acceder a determinada información relacionada con el estado del teléfono.
READ_EXTERNAL_STORAGE (maxSdk 18)	Permite leer información del almacenamiento externo. En este caso está limitado mediante maxSdkVersion="18".
READ_CALL_LOG	Permite acceder al registro de llamadas.
ACCESS_NETWORK_STATE	Permite consultar el estado de la conectividad de red.
ACCESS_COARSE_LOCATION	Permite acceder a la ubicación aproximada del dispositivo.

la aplicación solicita permisos muy superiores a los necesarios para una app bancaria (contactos, cuentas, llamadas, SMS, localización). Esto es una vulnerabilidad de sobre-privilegio: si la app se ve comprometida, hereda automáticamente la capacidad de leer contactos, geolocalizar al usuario, leer llamadas y enviar SMS — esta última capacidad, además, es explotada realmente por el propio código de la app.

Configuración insegura de la etiqueta <application>

<application android:allowBackup="true" android:debuggable="true" ...>

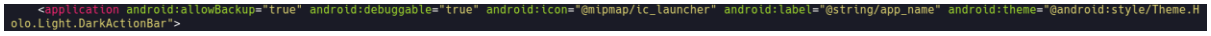


Figura 15. Etiqueta del manifest con allowBackup y debuggable activados.

Se identifican dos configuraciones inseguras:

• **android:debuggable="true":** la aplicación permite adjuntar un depurador en cualquier dispositivo sin necesidad de privilegios de root, lo que posibilita inspeccionar y manipular su ejecución en tiempo real. Esta bandera no debería estar activa en una compilación de producción.

• **android:allowBackup="true":** permite extraer el contenido completo del directorio privado de datos de la aplicación mediante adb backup, sin necesidad de root, exponiendo cualquier dato sensible almacenado localmente (credenciales, preferencias, bases de datos).

Componentes de la aplicación exportados sin protección

Se han identificado los siguientes componentes declarados con android:exported="true" y sin ningún atributo android:permission que restrinja su invocación:

Componente	Tipo
`PostLogin`	Activity
`DoTransfer`	Activity
`ViewStatement`	Activity
`ChangePassword`	Activity
`TrackUserContentProvider`	ContentProvider
`MyBroadCastReceiver`	BroadcastReceiver (acción personalizada `theBroadcast`)

Al no exigir ningún permiso, estos componentes son invocables por cualquier otra aplicación instalada en el dispositivo, sin necesidad de privilegios de root ni de pasar por el flujo de autenticación. Esto permite, entre otros escenarios: abrir directamente las pantallas posteriores al login, consultar el ContentProvider sin autenticación previa, o disparar el BroadcastReceiver enviando el broadcast theBroadcast desde una aplicación maliciosa.

A modo de contraste, se observa que el receptor EnableWalletOptimizationReceiver, perteneciente al SDK de Google Play Services, sí está correctamente configurado como android:exported="false", mientras que ninguno de los componentes propios de la lógica de negocio de la aplicación sigue esta buena práctica.

Análisis del código fuente de la aplicación en busca de vulnerabilidades

Una vez obtenido el código fuente de la aplicación mediante la decompilación del APK, se ha realizado una revisión de las principales clases que forman parte de la aplicación. El análisis se ha centrado especialmente en aquellas clases relacionadas con la autenticación, el almacenamiento de información sensible y los componentes accesibles desde otras aplicaciones.

Tras revisar el código, se han identificado dos vulnerabilidades relevantes: el almacenamiento inseguro de las credenciales mediante un mecanismo criptográfico débil y la exposición de un ContentProvider sin mecanismos de control de acceso.

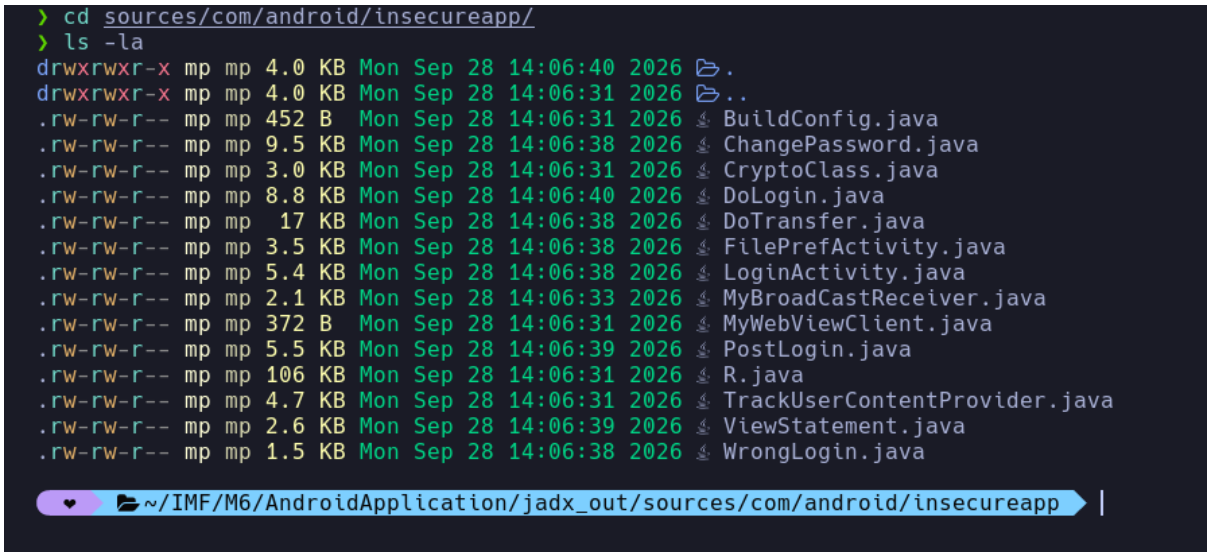


Figura 16. Listado de clases Java obtenidas tras la decompilación, punto de partida de la revisión de código.

Almacenamiento inseguro de credenciales mediante criptografía débil

Durante el análisis del código relacionado con el inicio de sesión se ha comprobado que la aplicación permite guardar las credenciales del usuario en las preferencias internas de Android.

En la clase CryptoClass.java se observa que la aplicación utiliza el algoritmo AES/CBC/PKCS5Padding para cifrar la información. Sin embargo, la clave utilizada para realizar el cifrado se encuentra directamente almacenada en el propio código de la aplicación. Además, se utiliza un vector de inicialización formado únicamente por ceros y definido de forma estática:

```
public class CryptoClass {
    String base64Text;
    byte[] cipherData;
    String cipherText;
    String plainText;
    String key = "This is the super secret key 123";
    byte[] ivBytes = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
```

```
Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
```

El principal problema no está en que se utilice AES, sino en la forma en la que se gestionan los elementos necesarios para el cifrado. La clave se encuentra hardcodeada en el código de la aplicación, por lo que cualquier persona que tenga acceso al APK y pueda analizarlo puede recuperar dicha clave. Del mismo modo, el IV utilizado es siempre el mismo.

Después, en la clase DoLogin.java, se observa que la contraseña introducida por el usuario es cifrada mediante la clase anterior y almacenada en las preferencias de la aplicación, en el método saveCreds():

```
    private void saveCreds(String username, String password) throws BadPaddingException, NoSuchPaddingException, IllegalBlockSizeException, NoSuchAlgorithmException, Inva
idKeyException, UnsupportedEncodingException, InvalidAlgorithmParameterException {
        SharedPreferences mySharedPreferences = DoLogin.this.getSharedPreferences("mySharedPreferences", 0);
        SharedPreferences.Editor editor = mySharedPreferences.edit();
        DoLogin doLogin = DoLogin.this;
        doLogin.rememberme_username = username;
        doLogin.rememberme_password = password;
        String base64Username = new String(Base64.encodeToString(doLogin.rememberme_username.getBytes(), 4));
        CryptoClass crypt = new CryptoClass();
        DoLogin doLogin2 = DoLogin.this;
        doLogin2.superSecurePassword = crypt.aesEncryptedString(doLogin2.rememberme_password);
        editor.putString("EncryptedUsername", base64Username);
        editor.putString("superSecurePassword", DoLogin.this.superSecurePassword);
        editor.commit();
    }
```

Figura 14. Almacenamiento de las credenciales del usuario en las preferencias de la aplicación.

También se observa que el nombre de usuario se almacena utilizando Base64. Este mecanismo no proporciona cifrado, sino únicamente una codificación del contenido.

El problema principal identificado es que, aunque la contraseña no se almacena directamente en texto plano, la protección aplicada puede ser recuperada a partir del propio código de la aplicación, ya que la clave utilizada para cifrarla forma parte del APK. Por tanto, un atacante que consiga acceder a los datos almacenados localmente

y disponga del APK podría obtener la clave presente en CryptoClass.java y utilizarla para descifrar la información almacenada por la aplicación.

- **Impacto**

Esta vulnerabilidad puede provocar la exposición de las credenciales almacenadas localmente. En caso de que un usuario utilice la opción de recordar sus credenciales, la contraseña queda almacenada en las preferencias de la aplicación bajo un mecanismo de protección que no resulta adecuado para proteger información de autenticación frente a un atacante con acceso al APK y al almacenamiento de la aplicación.

- **Recomendación**

No se debería incluir una clave criptográfica fija dentro del código de la aplicación. Además, las credenciales no deberían almacenarse directamente como parte de las preferencias de la aplicación. Como alternativa, se deberían utilizar los mecanismos seguros proporcionados por Android para almacenar información sensible y, cuando sea necesario almacenar secretos, utilizar claves gestionadas mediante mecanismos de almacenamiento seguro y algoritmos criptográficos adecuados.

ContentProvider exportado sin controles de acceso

La segunda vulnerabilidad encontrada está relacionada con el componente TrackUserContentProvider. En el AndroidManifest.xml se observa que este ContentProvider está declarado con android:exported="true" y sin ningún atributo android:permission, lo que hace que el proveedor pueda ser accesible desde otras aplicaciones instaladas en el mismo dispositivo:

```
        <provider android:authorities="com.android.insecureapp.TrackUserContentProvider" android:exported="true" android:name="com.android.insecureapp.TrackUserContentProvider
"/>
```

Posteriormente se ha revisado la implementación del componente en la clase TrackUserContentProvider.java. El proveedor utiliza la siguiente URI: ‘content://com.android.insecureapp.TrackUserContentProvider/trackerusers’ y permite realizar diferentes operaciones sobre la base de datos interna de la aplicación (consulta, inserción, actualización y eliminación):

```java
@Override // android.content.ContentProvider
public Uri insert(Uri uri, ContentValues values2) {
    long rowID = this.db.insert(TABLE_NAME, "", values2);
    if (rowID > 0) {
        Uri _uri = ContentUris.withAppendedId(CONTENT_URI, rowID);
        getContext().getContentResolver().notifyChange(_uri, null);
        return _uri;
    }
    throw new SQLException("Failed to add a record into " + uri);
}
```

```java
@Override // android.content.ContentProvider
public Cursor query(Uri uri, String[] projection, String selection, String[] selectionArgs, String sortOrder) {
    SQLiteQueryBuilder qb = new SQLiteQueryBuilder();
    qb.setTables(TABLE_NAME);
    if (uriMatcher.match(uri) == 1) {
        qb.setProjectionMap(values);
        if (sortOrder == null || sortOrder == "") {
            sortOrder = name;
        }
        Cursor c = qb.query(this.db, projection, selection, selectionArgs, null, null, sortOrder);
        c.setNotificationUri(getContext().getContentResolver(), uri);
        return c;
    }
    throw new IllegalArgumentException("Unknown URI " + uri);
}
```

```java
@Override // android.content.ContentProvider
public int update(Uri uri, ContentValues values2, String selection, String[] selectionArgs) {
    if (uriMatcher.match(uri) == 1) {
        int count = this.db.update(TABLE_NAME, values2, selection, selectionArgs);
        getContext().getContentResolver().notifyChange(uri, null);
        return count;
    }
    throw new IllegalArgumentException("Unknown URI " + uri);
}
```

```java
@Override // android.content.ContentProvider
public int delete(Uri uri, String selection, String[] selectionArgs) {
    if (uriMatcher.match(uri) == 1) {
        int count = this.db.delete(TABLE_NAME, selection, selectionArgs);
        getContext().getContentResolver().notifyChange(uri, null);
        return count;
    }
    throw new IllegalArgumentException("Unknown URI " + uri);
}
```

Durante la revisión del código no se ha observado ningún mecanismo de comprobación de permisos que limite el acceso a estas operaciones a la propia aplicación o a aplicaciones autorizadas. El problema se ve agravado por la configuración del AndroidManifest.xml, donde el proveedor está declarado como exported=true y no se especifica ningún permiso específico para acceder a él.

- **Impacto**

Un componente de este tipo puede convertirse en un punto de acceso a información de la aplicación desde procesos externos. Una aplicación instalada en el mismo dispositivo podría intentar acceder a la URI del proveedor y realizar operaciones sobre

los datos expuestos. Dependiendo de los permisos efectivos y de la configuración de la versión de Android utilizada, esto podría permitir consultar, insertar, modificar o eliminar información gestionada por el proveedor.

- **Recomendación**

La solución principal sería evitar la exposición del ContentProvider cuando no sea necesaria (android:exported="false"). En caso de que el proveedor tenga que ser accesible desde otras aplicaciones, debería implementarse un mecanismo de autorización adecuado mediante permisos específicos y comprobaciones de acceso antes de realizar las operaciones sobre la base de datos.

Análisis de aplicación móvil en dinámico

Tras el análisis estático, se ha procedido a analizar la aplicación en ejecución sobre el emulador, interceptando su tráfico de red y comprobando en el propio dispositivo el comportamiento descrito por el código fuente.

Interceptación del tráfico entre aplicación y servidor.

Se ha configurado Burp Suite como proxy de interceptación, en escucha en el puerto 8081 sobre todas las interfaces, y se ha configurado la aplicación para enviar su tráfico a través de dicho proxy:

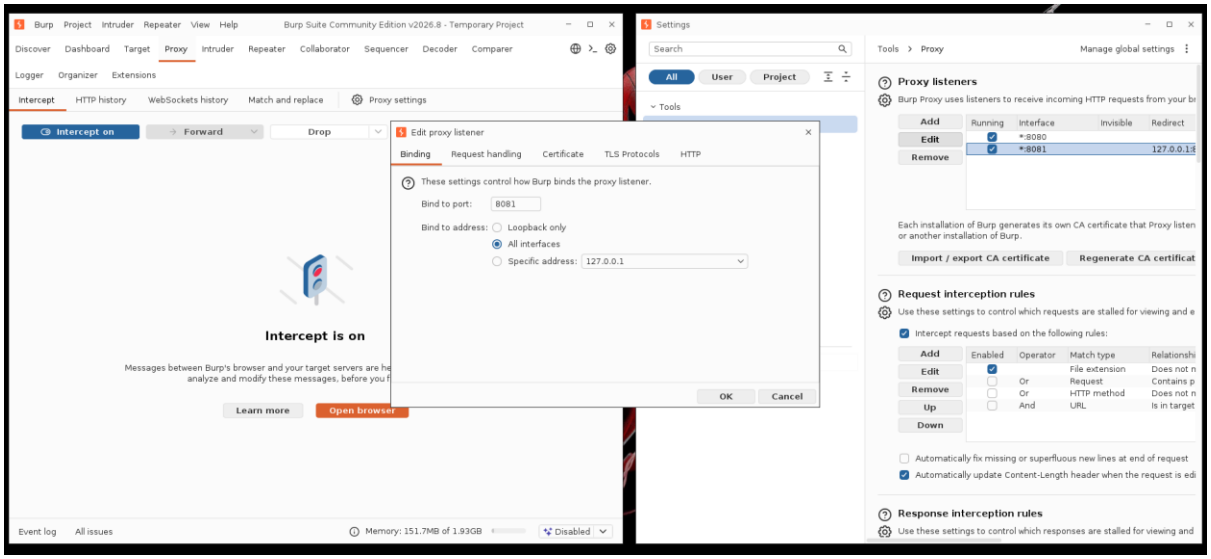


Figura 21. Configuración del listener de proxy de Burp Suite en el puerto 8081, accesible desde todas las interfaces de red.

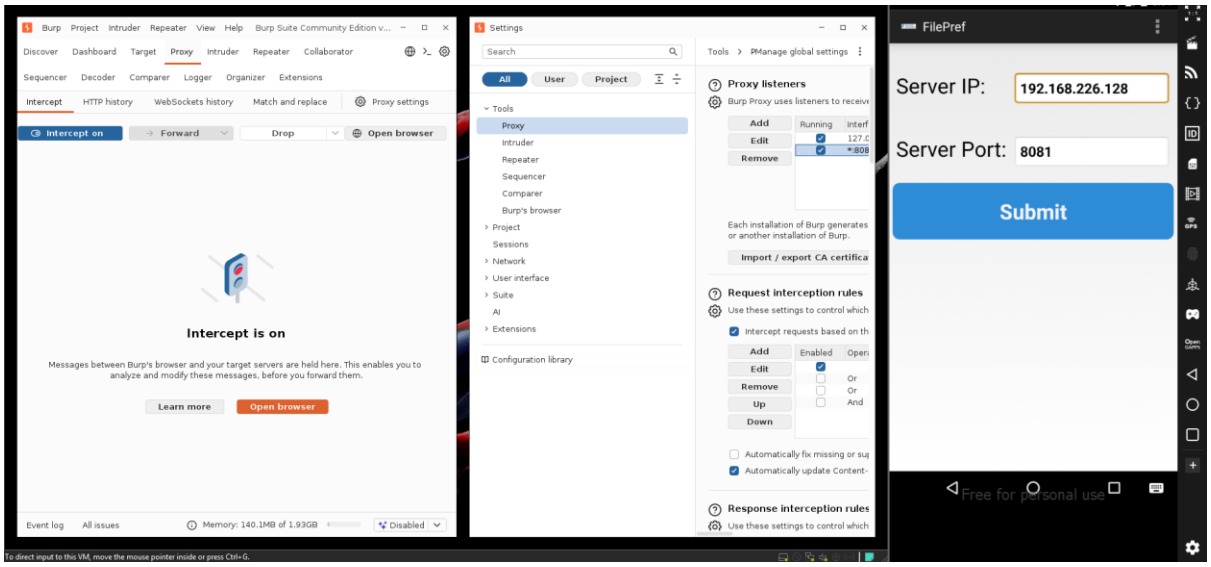


Figura 22. Interceptación activada en Burp Suite y configuración, en la propia aplicación, de la IP y el puerto del proxy

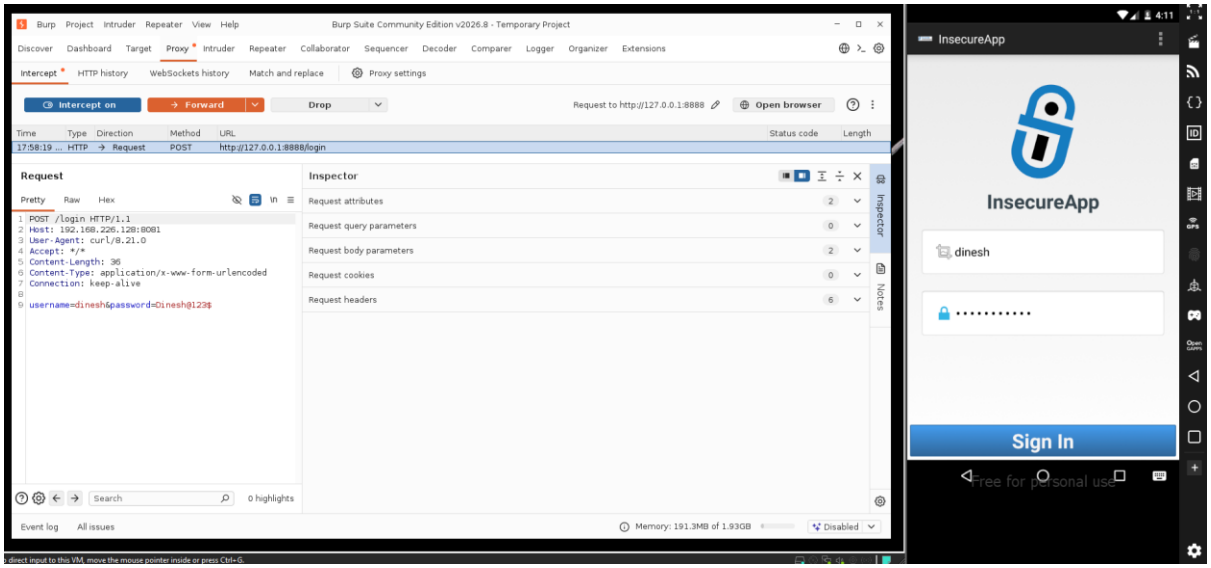


Figura 23. Petición POST /login interceptada en Burp Suite. Se observa que el usuario y la contraseña (username=dinesh&password=Dinesh@123$) viajan como parámetros en texto plano dentro del cuerpo de la petición.



Figura 24. Registro del servidor backend confirmando la recepción y validación correcta de las peticiones de login interceptadas.

Al no utilizarse HTTPS, cualquier atacante situado en la misma red (o capaz de interceptar el tráfico, como se ha hecho aquí con Burp Suite) puede capturar las credenciales del usuario en texto plano.

Vulnerabilidades de depuración de la aplicación.

Como ya se observó en el manifest, la aplicación se distribuye con android:debuggable="true". Se ha comprobado esta circunstancia directamente sobre el dispositivo:



Figura 25. dumpsys confirma los flags DEBUGGABLE, HAS_CODE, ALLOW_CLEAR_USER_DATA y ALLOW_BACKUP. Gracias a ello, run-as com.android.insecureapp permite acceder como el propio UID de la aplicación sin necesidad de root.

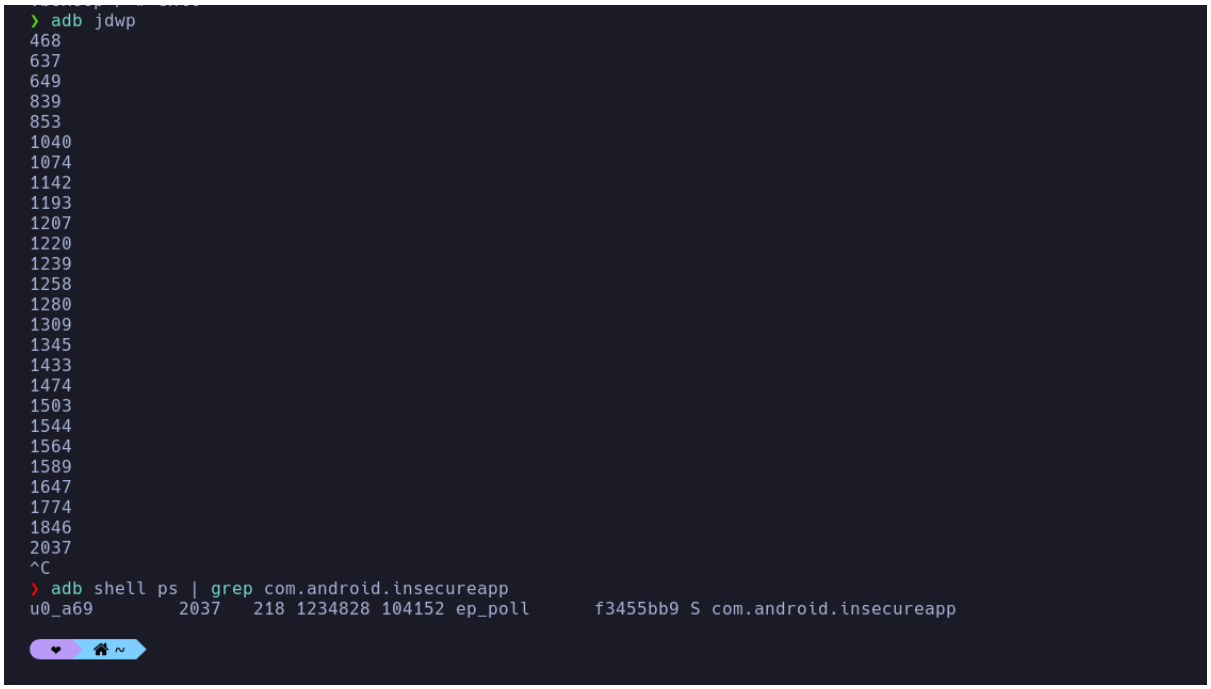


Figura 26. adb jdwp lista los PID de todos los procesos depurables del sistema, incluido el de InsecureApp (confirmado con adb shell ps), lo que permitiría adjuntar un depurador Java (JDWP) al proceso en cualquier dispositivo, sin privilegios de root.

- **Impacto**

Un atacante con acceso físico o por red al dispositivo puede adjuntar un depurador Java al proceso, inspeccionar variables en memoria, modificar el flujo de ejecución o saltarse comprobaciones del lado cliente. Combinado con allowBackup="true", también permite extraer el directorio de datos de la aplicación mediante adb backup sin necesidad de root.

- **Recomendación**

Deshabilitar android:debuggable y android:allowBackup en las compilaciones de producción, asegurando que la variante debug nunca se publica, y valorar la incorporación de mecanismos adicionales de detección de depuración/anti-tampering en aplicaciones que manejan información financiera.

Almacenamiento inseguro de datos.

Aprovechando que la aplicación es depurable, se ha utilizado run-as para inspeccionar el directorio privado de datos de la aplicación sin necesidad de un dispositivo rooteado:

```
vbox86p:/ # run-as com.android.insecureapp find . -maxdepth 3 -type f
./databases/mydb
./databases/mydb-journal
./shared_prefs/com.android.insecureapp_preferences.xml
vbox86p:/ # run-as com.android.insecureapp ls -la shared_prefs
total 24
drwxrwx--x 2 u0_a69 u0_a69 4096 2026-10-01 17:38 .
drwx------ 6 u0_a69 u0_a69 4096 2026-10-01 17:33 ..
-rw-rw---- 1 u0_a69 u0_a69  167 2026-10-01 17:38 com.android.insecureapp_preferences.xml
```

Figura 27. Directorio de datos privados de la aplicación (cache, code_cache, databases, shared_prefs), accesible mediante run-as gracias a debuggable="true".

```
vbox86p:/ # run-as com.android.insecureapp cat shared_prefs/com.android.insecureapp_preferences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="serverport">8081</string>
    <string name="serverip">192.168.226.128</string>
</map>
```

Figura 28. Contenido del fichero com.android.insecureapp_preferences.xml en shared_prefs: la IP y el puerto del servidor se almacenan en texto plano.

En este caso concreto, el fichero de preferencias únicamente contenía los parámetros de conexión al servidor en texto plano, sin cifrar. Como se ha visto en el apartado 3.1, el propio código de DoLogin.java almacena también en SharedPreferences el usuario y la contraseña cifrada con una clave hardcodeada y recuperable del propio APK cuando el usuario marca la opción de recordar credenciales; ese valor se guarda en esta misma ubicación (shared_prefs) y heredaría la misma falta de protección real frente a un atacante con acceso al dispositivo y al APK.

- **Impacto**

Cualquier dato almacenado en shared_prefs o en la base de datos local es accesible sin privilegios de root gracias a debuggable="true", y extraíble por mediante adb backup gracias a allowBackup="true". En un dispositivo comprometido o perdido, esto expone tanto la configuración de red como, potencialmente, las credenciales cifradas de forma débil del apartado 3.1.

- **Recomendación**

No almacenar en SharedPreferences información sensible sin cifrar fuertemente (usar EncryptedSharedPreferences respaldado por Android Keystore), y limitar al mínimo imprescindible los datos persistidos localmente por la aplicación.

Bypass de autenticación.

- Se ha comprobado el comportamiento del endpoint de login del backend ante una petición con credenciales incorrectas:

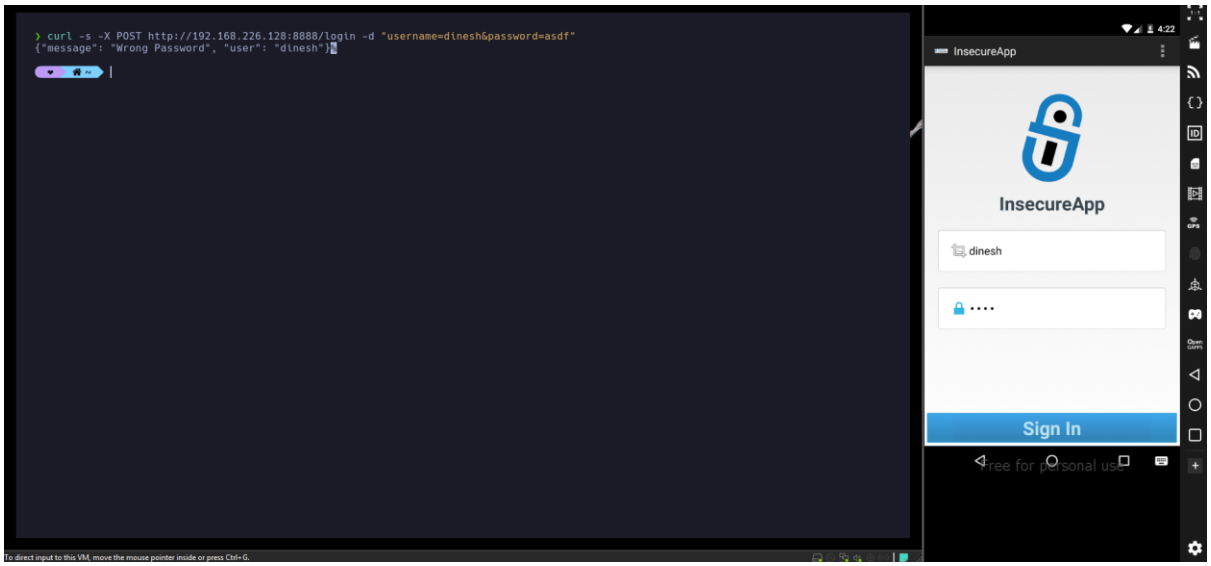


Figura 29. Petición normal al endpoint /login con una contraseña incorrecta: el servidor responde "Wrong Password".

- Durante la revisión del backend se ha detectado la existencia de un segundo endpoint, /devlogin, que replica la funcionalidad de /login pero sin validar realmente la contraseña recibida:



Figura 30. El mismo usuario (dinesh) con una contraseña incorrecta es aceptado como válido al usar el endpoint /devlogin en lugar de /login.

- Refuerza el PoC probando con un usuario que ni siquiera existe



Figura 31. Prueba de concepto reforzada: incluso un usuario que no existe en el sistema (asdf) es aceptado como "Correct Credentials" por /devlogin, confirmando que el endpoint no realiza ninguna validación real.

- **Impacto**

Se trata de una vulnerabilidad crítica: un backdoor de autenticación que permite a cualquier atacante autenticarse como cualquier usuario (existente o no) sin conocer su

contraseña, obteniendo acceso completo a la funcionalidad de la aplicación, incluidas las transferencias y la consulta de extractos vistas como componentes exportados en el apartado 2.

- **Recomendación**

Eliminar por completo los endpoints de depuración/backdoor (/devlogin) del código que llega a producción. Se recomienda separar claramente los entornos de desarrollo y producción y someter el código a revisión antes de cada despliegue para evitar que este tipo de accesos queden expuestos.

Bypass de detección de root.

En el código de PostLogin.java se ha localizado la lógica encargada de comprobar si el dispositivo está rooteado, basada en la existencia del APK de Superuser y en la presencia del binario su en el PATH del sistema:



Figura 32. Comprobación de root en PostLogin.java: isrooted = doesSuperuserApkExist("/system/app/Superuser.apk") || doesSUexist().

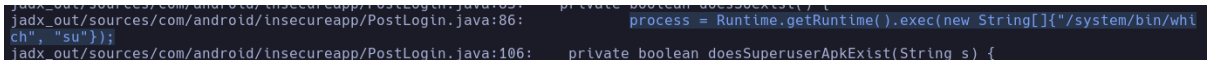


Figura 33. Implementación de doesSUexist(): ejecuta Runtime.getRuntime().exec({"which","su"}) y comprueba si obtiene una ruta válida.

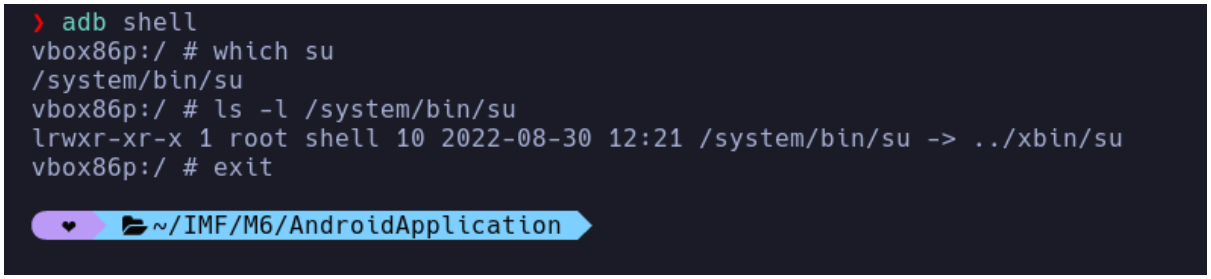


Figura 34. Comprobación manual en el dispositivo: which su localiza el binario en /system/bin/su, enlazado a /system/xbin/su.

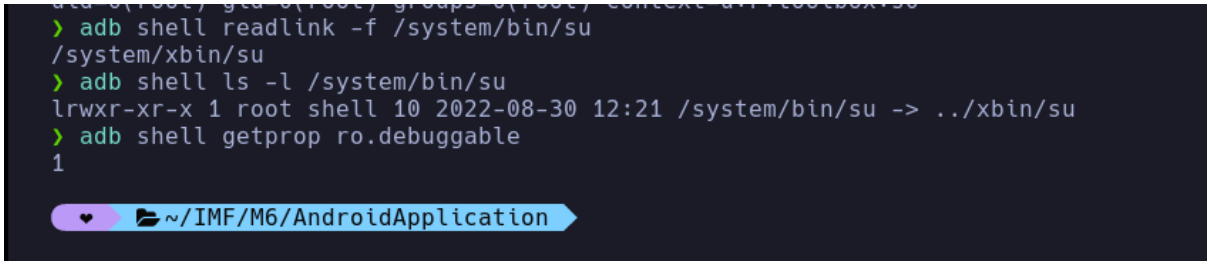


Figura 35. Confirmación de los permisos del binario su (setuid root) y de la propiedad ro.debuggable=1 del sistema.

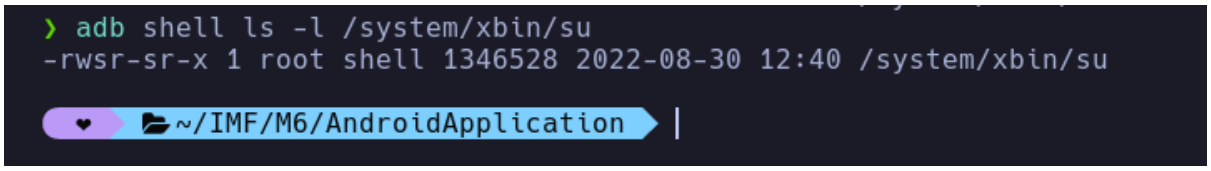


Figura 36. El binario real en /system/xbin/su presenta permisos -rwsr-sr-x propiedad de root



Figura 37. adb shell su -c 'whoami' devuelve root, confirmando acceso privilegiado efectivo en el dispositivo de pruebas.



Figura 38. adb shell su -c id y adb shell id muestran que, en este dispositivo Genymotion, el propio shell de adb ya se ejecuta con uid=0(root) por defecto.

- **Impacto**

Una detección de root basada únicamente en comprobar la existencia de un binario su en rutas conocidas es fácilmente evadible, por lo que no constituye una protección real

frente a un atacante que ejecute la aplicación en un dispositivo rooteado para manipular su comportamiento.

- **Recomendación**

Combinar varias técnicas de detección de root (comprobación de propiedades del sistema, rutas adicionales, integridad del propio APK, detección de hooking de Frida/Xposed) y no basar decisiones de seguridad críticas únicamente en comprobaciones del lado cliente, que siempre pueden ser manipuladas en un dispositivo bajo control del atacante.

Bypass de detección de emulador.

De forma análoga a la detección de root, PostLogin.java implementa una comprobación para detectar si la aplicación se está ejecutando sobre un emulador, basada en propiedades de compilación:

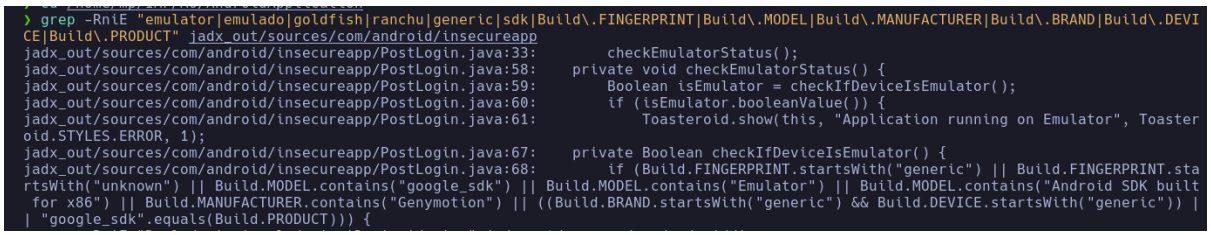


Figura 40. checkIfDeviceIsEmulator(): la detección se basa en Build.FINGERPRINT, Build.MODEL, Build.MANUFACTURER, Build.BRAND, Build.DEVICE y Build.PRODUCT, buscando cadenas típicas de emuladores ("generic", "google_sdk", "Emulator", "Genymotion"...).

Se han consultado las propiedades reales del dispositivo virtual utilizado para la auditoría:

```
❯ adb shell getprop ro.build.fingerprint
google/vbox86p/vbox86p:8.0.0/OPR6.170623.017/434:userdebug/test-keys
❯ adb shell getprop ro.product.model
Pixel
❯ adb shell getprop ro.product.manufacturer

❯ adb shell getprop ro.product.brand
Google
❯ adb shell getprop ro.product.device
vbox86p
❯ adb shell getprop ro.product.name
vbox86p

❤  ~/IMF/M6/AndroidApplication
```

Figura 41. Propiedades del sistema del emulador: fingerprint google/vbox86p/vbox86p:8.0.0/.../test-keys, model=Pixel, manufacturer=Google, brand=Google, device=vbox86p, name=vbox86p.

Ninguna de estas propiedades coincide con las cadenas comprobadas por el código. El perfil de Google Pixel que emplea Genymotion por defecto evade así, de forma natural, la comprobación implementada, sin necesidad de ninguna manipulación adicional.



Figura 42. Logcat filtrado por "Application running|Emulator|Real device|insecureapp" durante el arranque de la app: no aparece en ningún momento el Toast "Application running on Emulator", confirmando que la detección no se ha activado pese a ejecutarse sobre un emulador.

- **Impacto**

La detección de emulador implementada depende de una lista cerrada de valores de Build.* fácilmente evadibles, que ni siquiera cubre perfiles de emuladores habituales como el Google Pixel por defecto de Genymotion. Esto permite ejecutar y analizar la aplicación en un entorno controlado (emulador) sin activar ninguna contramedida, facilitando la instrumentación, la depuración y la automatización de ataques descritos en apartados anteriores.

- **Recomendación**

No basar la detección de emulador únicamente en una lista estática de valores Build.*; complementarla con comprobaciones adicionales y, en cualquier caso, no depender exclusivamente de controles del lado cliente para decisiones de seguridad relevantes.